ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

Année universitaire 2026-2027

TP06

Automatiser les workflows MLOps

Orchestration avec Prefect
et intégration continue avec GitHub Actions

Réalisé par	BENHEDDANE Mohamed
Filière et groupe	5IASD - G8
Module	Déploiement d'Applications Intelligentes
Encadrement	Pr. Hafssa NACIRI
Date des essais	9 novembre 2026

Dépôt public

github.com/BENHEDDANEMed/mlops-tp6-workflows

Le pipeline du TP3 est conservé. Le workflow accepte le modèle à F1 = 0,8423, refuse le modèle dégradé à F1 = 0,25, puis réussit après restauration.

01 / COMPTE RENDU DU TP06

Objectif et environnement

Le TP3 fournissait un pipeline reproductible avec DVC. Le TP6 ajoute l'exécution observable de ce pipeline, son contrôle de qualité, une planification hebdomadaire et une vérification automatique à chaque modification du code.

Élément	Configuration utilisée
Poste local	Windows / PowerShell, Python 3.14.7
Orchestration	Prefect 3.9.2, serveur http://127.0.0.1:4200
Pipeline	DVC 3.67.1, 7 000 transactions v2, séparation train/test 75 % / 25 %
Modèle	RandomForestClassifier, 300 arbres, profondeur 12, graines 42
Qualité	F1 ≥ 0,82, seuil lu dans params.yaml
CI	GitHub Actions, ubuntu-latest, Python 3.12

Architecture mise en place

Le serveur conserve les déploiements, les états et les journaux. Le processus **serve** exécute le déploiement simple. Pour le second déploiement, **pool-tp** organise les demandes et un **worker de type process** les exécute sur le poste.

Préparation et compatibilité

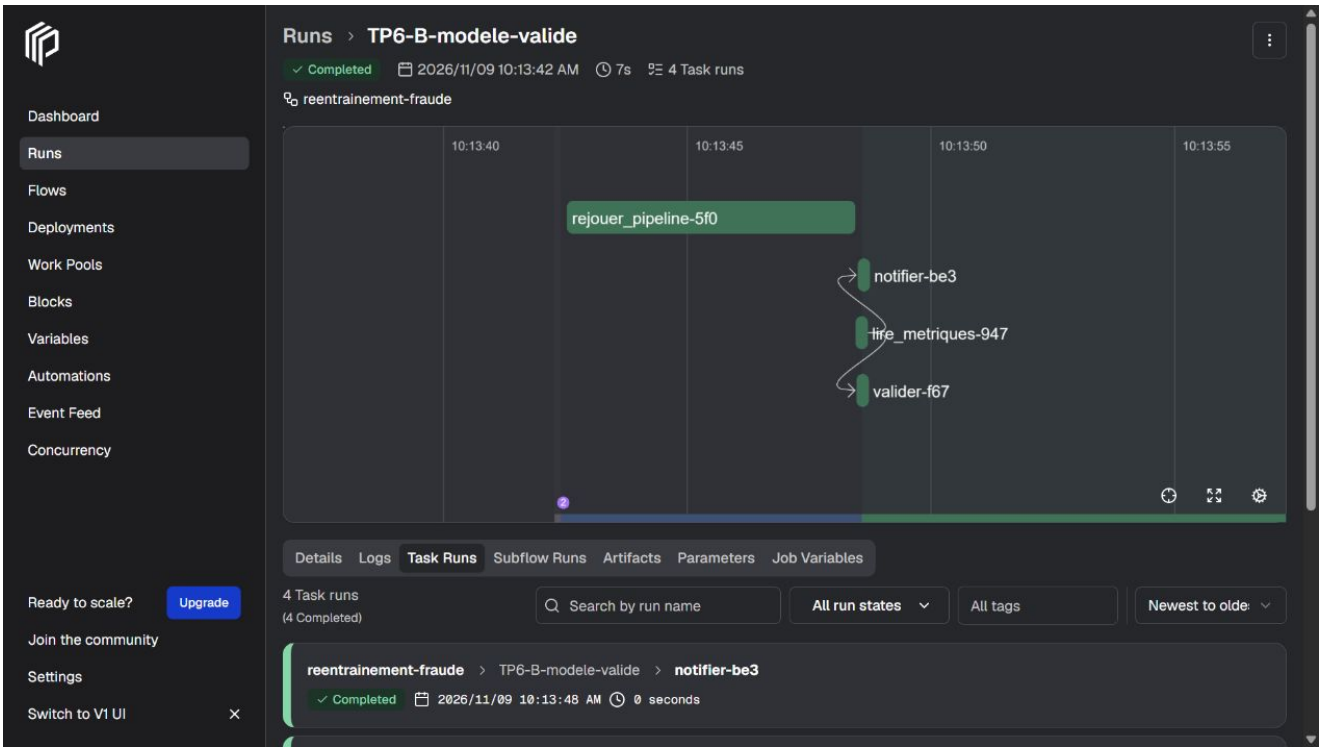
Le dossier existant reprend l'historique du TP3 et ses tags v1 et v2. L'ajout de Prefect n'a créé aucun conflit : pip check indique qu'aucune dépendance n'est cassée. MLflow 3.18.0 reste installé mais n'intervient pas dans ce TP.

Au premier dvc status, les trois scripts apparaissaient modifiés à cause des fins de ligne CRLF du clonage Windows. Un dvc repro a actualisé leurs empreintes dans dvc.lock. Les scripts sont inchangés dans Git et le F1 reste 0,8423.

02 / COMPTE RENDU DU TP06

Un workflow en quatre tâches

flow_mlops.py appelle successivement **rejouer_pipeline**, **lire_metriques**, **valider** et **notifier**. La première tâche appelle le DVC du même environnement Python ; le pipeline de préparation, entraînement et évaluation reste dans dvc.yaml.



Capture 1 - Exécution TP6-B-modele-valide : quatre tâches Completed.

Résultat mesuré

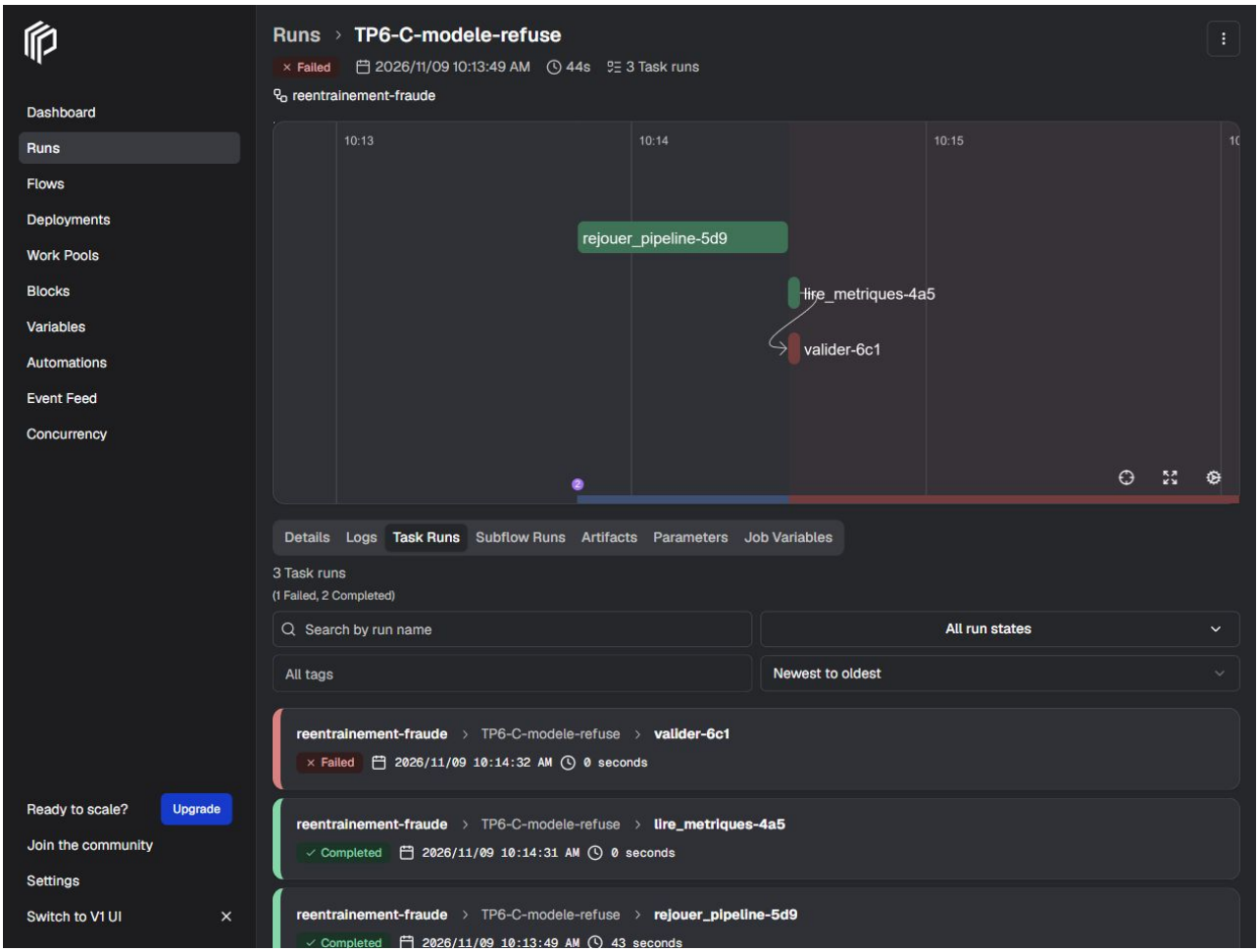
F1	Précision	Rappel	Décision
0,8423	0,8698	0,8166	Accepté : F1 ≥ 0,82

Sans modification des dépendances, DVC ignore les étapes déjà à jour. La première exécution Prefect dure environ sept secondes. La notification du sujet est une ligne dans les journaux, sans envoi à un destinataire externe.

03 / COMPTE RENDU DU TP06

La règle de qualité arrête le flux

Le paramètre max_depth a été temporairement abaissé de 12 à 2. DVC a recalculé l'entraînement et l'évaluation ; la validation a refusé le nouveau F1.



Capture 2 - Le run échoue dans valider ; seules trois tâches existent, sans notifier.

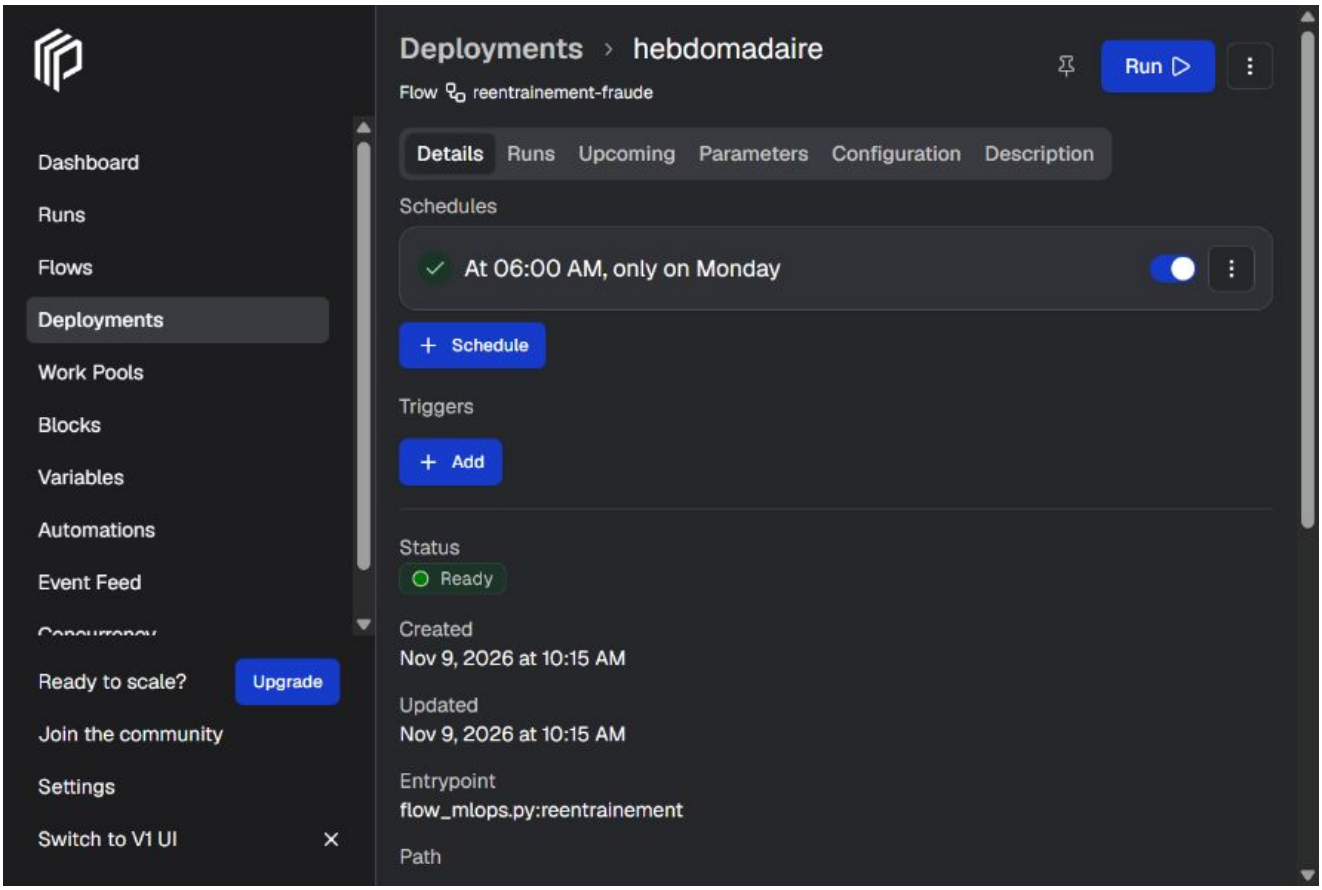
État du modèle	F1	État Prefect	Code du contrôle CI
Normal : profondeur 12	0,8423	Completed	0
Dégradé : profondeur 2	0,25	Failed	1
Restauré : profondeur 12	0,8423	Completed	0

L'API confirme trois tâches pour le run refusé : le pipeline et la lecture réussissent, valider échoue, notifier n'est jamais créée. Après restauration, params.yaml et reports/metrics.json sont identiques à leur état initial. Seule la normalisation initiale des empreintes CRLF subsiste dans dvc.lock.

04 / COMPTE RENDU DU TP06

Planifier le lundi à 6 h

planifier.py sert le déploiement **hebdomadaire**. L'expression cron **0 6 * * 1** fixe le lundi à 6 h. Le fuseau **Africa/Casablanca** est explicite pour éviter l'interprétation UTC par défaut du court exemple du sujet.



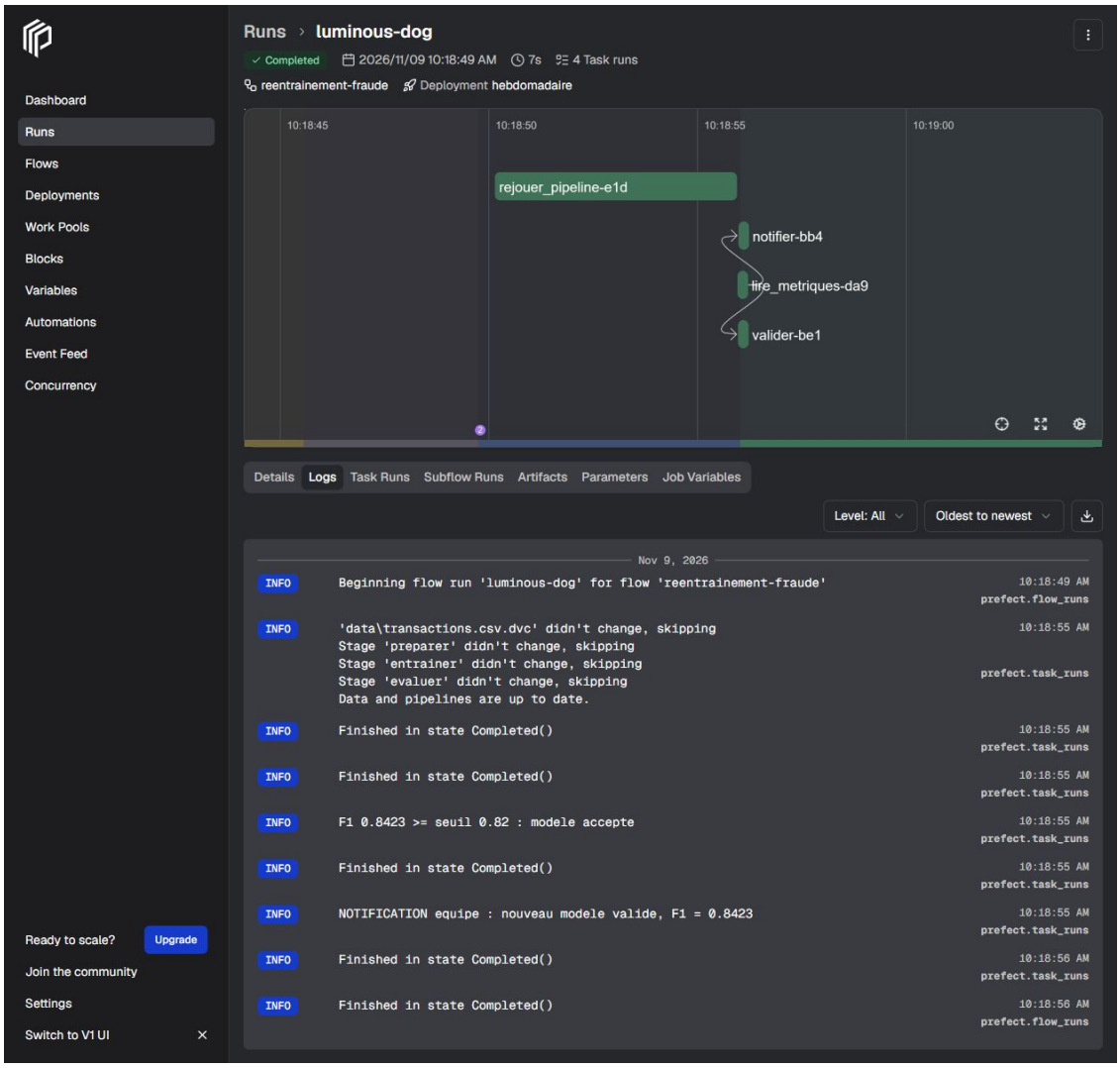
Capture 3a - Déploiement hebdomadaire, planification active le lundi à 6 h.

Le serveur et le processus serve doivent rester actifs. La planification enregistrée ne suffit pas à elle seule à exécuter le code sur une machine arrêtée.

05 / COMPTE RENDU DU TP06

Déclencher depuis l'interface

Dans Deployments, le bouton Run puis Quick run a créé le run **luminous-dog**. Le processus serve a exécuté les quatre tâches et obtenu le même F1 de 0,8423.



Capture 3b - Quick run lancé depuis l'interface : état Completed et validation réussie.

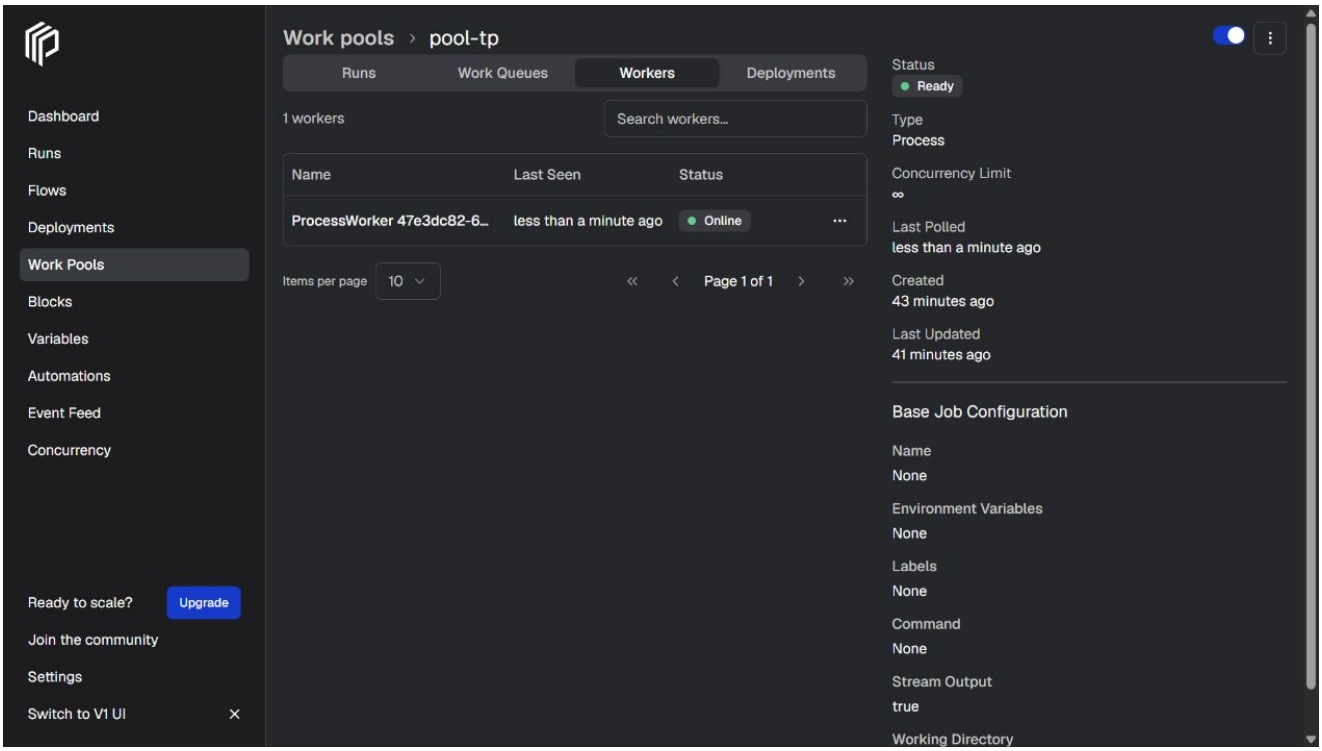
Essai après arrêt de serve

Après Ctrl+C sur serve, un second Quick run, golden-beagle, n'a jamais démarré. Il est passé en état Late, sans heure de début et sans journaux. Le worker de pool-tp ne prend pas ce run, car ce déploiement n'est pas associé au pool. La preuve a été conservée dans essai_sans_serve.json, puis cette exécution de test a été supprimée.

06 / COMPTE RENDU DU TP06

Séparer serveur, pool et worker

Le déploiement **hebdomadaire-pool** utilise **pool-tp**, de type process. prefect.yaml contient la planification, la limite d'une exécution simultanée et le répertoire local à charger avec set_working_directory.



Capture 4a - Work pool pool-tp et worker actif sur le poste.

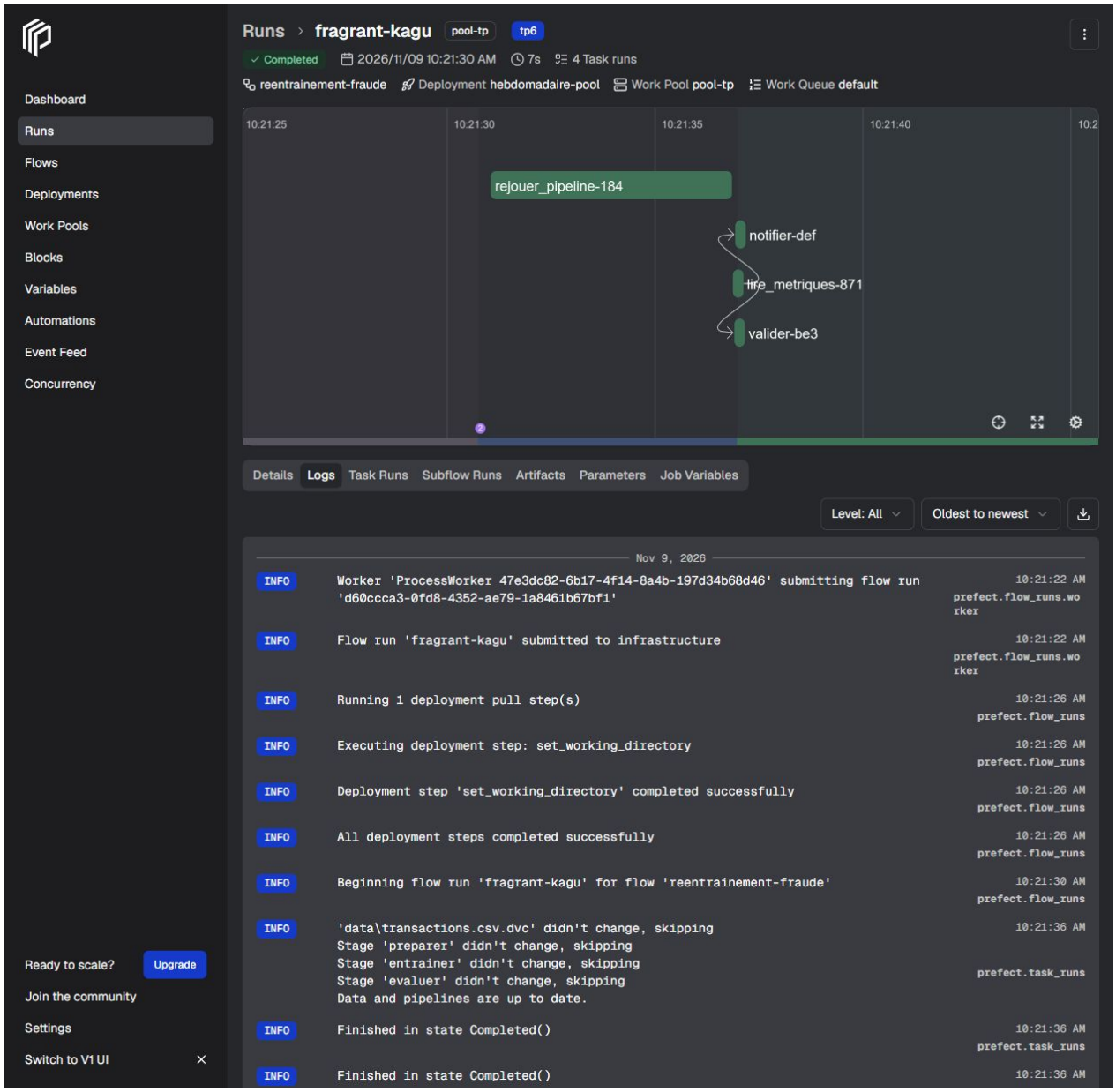
Exécution contrôlée

Un Quick run a créé **fragrant-kagu**. Le worker a chargé le dossier du projet, exécuté les quatre tâches puis terminé en **Completed**. Le journal indique F1 = 0,8423, et l'API rattache ce run au déploiement hebdomadaire-pool et à pool-tp.

07 / COMPTE RENDU DU TP06

Observer l'exécution par le worker

Le journal distingue la soumission du run par le worker, le chargement du répertoire de travail et l'exécution du flow. Les quatre tâches terminent avec succès.



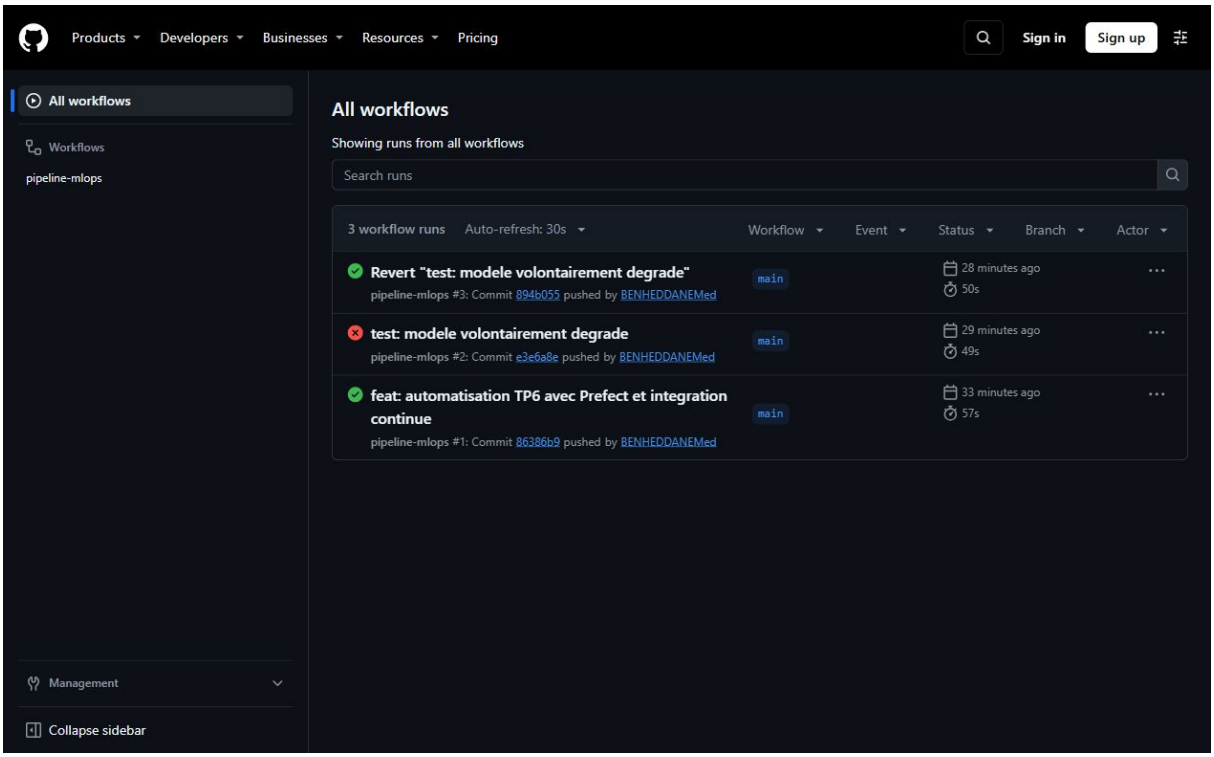
Capture 4b - Exécution réalisée par le worker, terminée avec succès.

Cette exécution vérifie toute la chaîne serveur / pool / worker. Le serveur enregistre la demande ; le worker disponible l'exécute sur le poste. La tâche de validation lit le même seuil de 0,82 et accepte le F1 mesuré de 0,8423.

08 / COMPTE RENDU DU TP06

Vérifier chaque push avec GitHub Actions

Le dépôt public mlops-tp6-workflows conserve l'historique du TP3 et ses tags. L'ancien lien distant a été renommé tp3 ; origin pointe vers le nouveau dépôt. Le workflow pipeline-mlops installe les dépendances de la CI, régénère les données v2, exécute DVC, contrôle le F1 puis conserve les rapports, même en cas d'échec.



Capture 5a - Historique réel des exécutions GitHub Actions.

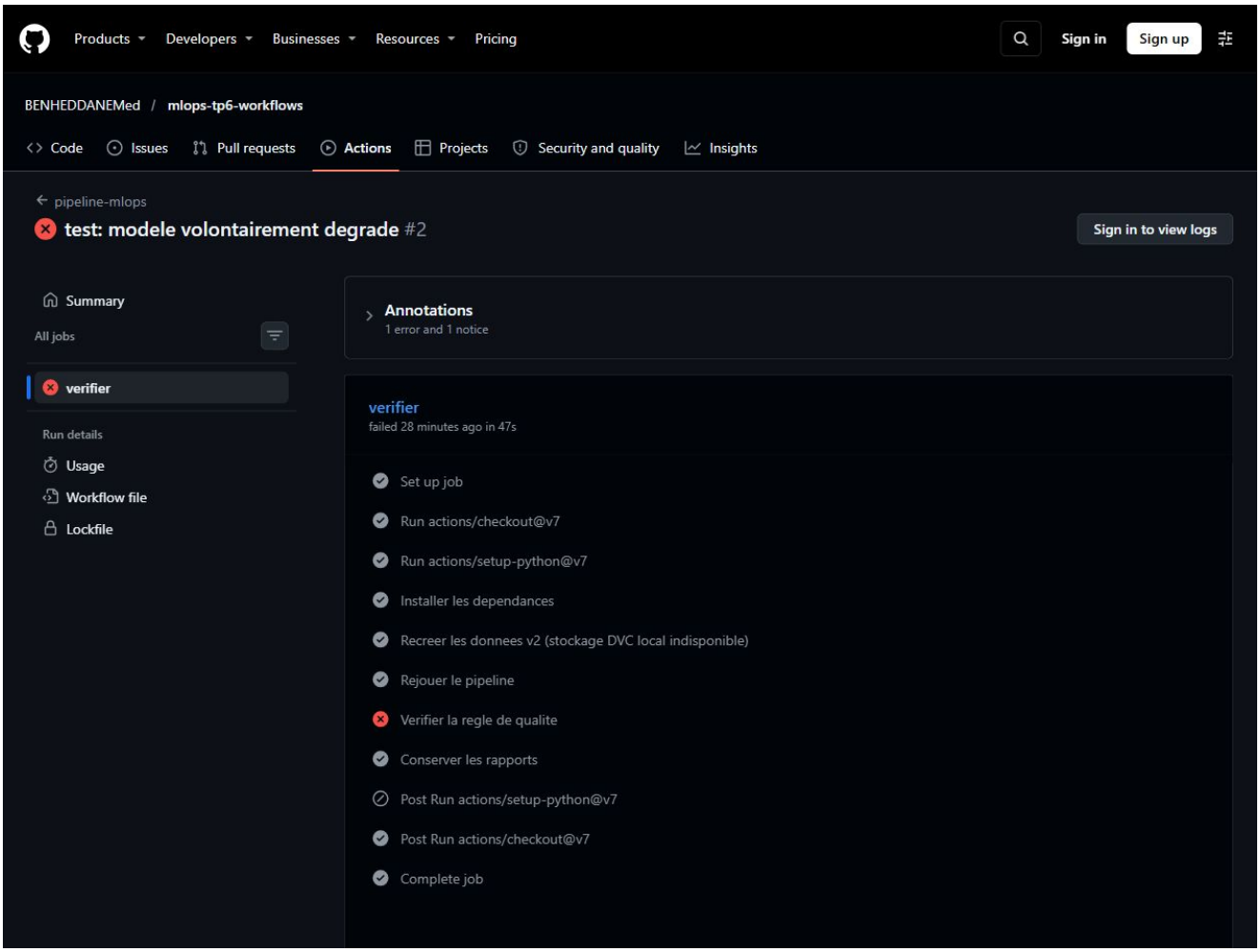
Run	Commit	Résultat	F1	Étape de qualité
1	86386b99	success	0.8423	success
2	e3e6a8e2	failure	0.25	failure
3	894b0553	success	0.8423	success

Le test rouge modifie uniquement max_depth à 2. Il échoue sur la règle de qualité. Un **git revert --no-edit** crée ensuite un nouveau commit et restaure la profondeur 12, sans réécrire l'historique. Les journaux et les artefacts donnent les F1 réellement obtenus.

Les actions checkout, setup-python et upload-artifact utilisent les versions majeures v7 documentées au moment de la vérification. Les versions numériques sont figées dans requirements-ci.txt pour limiter les différences de calcul.

09 / COMPTE RENDU DU TP06

Comprendre le refus en CI



Capture 5b - Échec volontaire de la règle de qualité dans GitHub Actions.

La CI reconstruit les données à partir de donnees.py et nouvelles_donnees.py, avec les graines 7 et 8. Elle n'accède pas au dossier DVC du poste. Les sorties du run sont conservées comme artefact rapports ; le F1 du modèle refusé vaut 0,25.

Extrait vérifié du journal GitHub téléchargé

F1 = 0.25 | seuil = 0.82
ECHEC : F1 0.25 < seuil 0.82 : modele refuse
Process completed with exit code 1.

La restauration finale est vérifiée dans Git et sur GitHub : le run 3 réussit avec max_depth = 12, F1 = 0,8423 et un contrôle de qualité accepté.

10 / COMPTE RENDU DU TP06

Synthèse : deux outils complémentaires

Critère	Prefect	GitHub Actions
Déclenchement	Cron hebdomadaire, Quick run, API	Push ou workflow_dispatch
Lieu d'exécution	Poste de serve ou du worker	Runner Linux éphémère de GitHub
Surveillance	États et journaux dans le serveur Prefect	Actions, résultat lié au commit et rapports
Échec	Deux réessais pour DVC ; refus immédiat par valider	Code non nul : job en échec ; rapports conservés
Usage adapté	Traitements récurrents et réentraînement supervisé	Validation automatique d'une modification
Versionné dans Git	Flow Python, planifier.py, prefect.yaml, pipeline et paramètres	ci.yml, dépendances CI, génération et contrôle qualité

Conclusion en cinq lignes

La CI valide chaque modification avant son utilisation opérationnelle.

L'orchestrateur exécute et surveille les traitements récurrents.

La même règle F1 bloque l'acceptation d'un modèle insuffisant.

Les journaux et les rapports permettent de diagnostiquer les échecs.

Prefect et GitHub Actions se complètent et ne se remplacent pas.

11 / COMPTE RENDU DU TP06

Réponses aux questions

1. Deux rôles

Un orchestrateur planifie et exécute les traitements récurrents, suit leurs dépendances et conserve leurs états. La CI vérifie une modification du code dans un environnement reconstruit. Ici, Prefect pilote le réentraînement ; GitHub Actions rejoue le pipeline à chaque push et refuse un F1 insuffisant.

2. Lundi, 6 h du matin

L'équipe retrouve dans Prefect le run Failed, l'heure, la tâche fautive, son message d'erreur et les journaux DVC. Lors de notre essai, valider signale F1=0,25 < 0,82 et notifier ne s'exécute pas. La fonction notifier du sujet écrit uniquement un message de succès dans les journaux : elle n'envoie aucun e-mail et ne prévient personne d'un échec. Pour être avertie à 6 h sans consulter l'interface, une vraie équipe doit ajouter une automatisation d'alerte sur les états Failed/Crashed ou les exécutions en retard, avec un canal de notification configuré et testé. Le TP local ne configure pas cette alerte externe.

3. Les réessais

Une erreur temporaire du pipeline peut disparaître lors d'un nouvel essai : indisponibilité d'un fichier ou d'une ressource, par exemple. Deux réessais espacés de 10 secondes offrent cette possibilité, pour un maximum de trois tentatives. En revanche, relire le même F1 avec le même seuil ne peut pas améliorer le modèle : réessayer la validation retarderait inutilement le refus. Il faut corriger le modèle ou les données, puis relancer.

12 / COMPTE RENDU DU TP06

Réponses et références

4. Interface ou fichier

L'interface facilite la lecture des états, le déclenchement manuel et la consultation des journaux. Des changements faits uniquement dans l'interface peuvent toutefois diverger de Git, être difficiles à revoir et disparaître lors d'un redéploiement. Un fichier versionné fournit un historique, une revue et une configuration reproductible. Ici, les définitions Python et prefect.yaml sont la référence ; les essais sont lancés et observés dans l'interface.

5. Un autre déclencheur

L'arrivée d'un lot suffisamment grand de nouvelles transactions, ou une dérive des distributions des variables, peut déclencher un réentraînement. Un traitement de surveillance peut comparer les nouveaux lots à une référence, calculer une mesure de dérive et émettre un événement lorsque le seuil est dépassé. Cet événement peut appeler l'API de déploiement Prefect. Il faut dédupliquer les événements et limiter la fréquence des lancements. Cette piste est une proposition, pas une fonctionnalité exécutée dans ce TP.

6. Les données de la CI

Le stockage DVC est un dossier local du poste et n'existe pas sur le runner Linux de GitHub. Les scripts donnees.py puis nouvelles_donnees.py reconstruisent les 7 000 transactions v2 avec les graines 7 et 8, sans publier les données brutes dans Git. Dans une vraie équipe, on utilise un stockage distant DVC partagé et accessible par le runner, avec authentification à privilèges limités via les secrets GitHub ou une identité fédérée. On versionne les tickets .dvc et dvc.lock et on contrôle l'accès aux données sensibles. Les fins de ligne CSV peuvent différer entre Windows et Linux ; DVC peut donc recalculer les étapes malgré les mêmes données numériques.

Références et traçabilité

Sujet : TP06_Automatiser_Workflows_MLOps.pdf, année 2026-2027. Les résultats proviennent des exports JSON Prefect et GitHub Actions du projet, des métriques et des captures réelles des interfaces.

https://reference.prefect.io/prefect/flows/

https://github.com/actions/checkout

https://github.com/actions/setup-python

https://github.com/actions/upload-artifact